JAVA SDE-2 INTERVIEW PREPARATION SERIES

JAVA ENGINEERING - BOOK 03 OF 09 - STUDY STEP 19B

Maven and Gradle for Java Engineers

From First Build to Reproducible Delivery and SDE-2 Diagnostics

BY

Vinay Reddy Kalluri

A concept-first, side-by-side guide to Maven and Gradle lifecycles, dependency graphs, tests, modules, artifacts, secure delivery, performance, migration, and complex build incidents.

MAVEN | GRADLE | DEPENDENCIES | CI | SECURE BUILDS | SDE-2

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-29

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `JAVA 02 — Git and GitHub`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Teach one durable Java build model first, map Maven and Gradle to it precisely, and develop the diagnostic and delivery judgment expected in SDE-2 interviews and production ownership.

Readiness map

Decision	Evidence
START HERE	A Java build works in one environment but not another; Dependency resolution, classpaths, lifecycle, or task-graph behavior is unclear; An interview requires Maven-versus-Gradle mechanics and trade-offs; A team must secure, accelerate, publish, migrate, or recover a build.
PREPARE FIRST	Java Foundations Book 01; Git and GitHub Book 02 recommended; Java 21 and a terminal for the executable labs.
MOVE ON WHEN	Explain the shared build contract before choosing Maven or Gradle syntax; Operate Maven lifecycle, effective model, reactor, scopes, testing, and publication with evidence; Operate Gradle lifecycle, tasks, configurations, variants, caches, multi-project builds, and publication with evidence; Diagnose dependency, environment, packaging, cache, CI, security, and migration failures at SDE-2 depth.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Java Segment Position

JAVA 02 - Git and GitHub	**YOU ARE HERE** **JAVA 03 - Maven and Gradle**	JAVA 04 - Advanced Java B - Language

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Build Foundations and the Learning Path

A Java build is a repeatable transformation from reviewed source code to verified artifacts. Maven and Gradle automate that transformation, but the tool is not the model. Before learning XML or Kotlin DSL, learn the questions every build must answer.

Learning objectives

By the end of this chapter, you should be able to:

- distinguish source, dependency, task, output, artifact, and publication;
- explain why an IDE build is not a team build contract;
- draw a build as a dependency graph rather than a command list;
- identify the responsibilities shared by Maven and Gradle;
- follow the book from first build through SDE-2 incident diagnosis.

The build contract

Consider a pricing service. Its repository contains Java source, tests, resources, and build configuration. A useful build contract states:

1. which tool and JDK execute the build;
2. which source files and generated inputs participate;
3. which external components are resolved;
4. which checks must pass;
5. which artifacts are produced;
6. which environment inputs are permitted;
7. how another machine reproduces the result.

```text
SOURCE + CONFIGURATION + TOOLCHAIN + DEPENDENCIES
                         |
                         v
              ordered, verified work
                         |
           +-------------+-------------+
           |             |             |
           v             v             v
       classes       reports      JAR / metadata
```

If a developer can build only by clicking an undocumented IDE action, the process is not yet a reliable team contract. IDEs should import and delegate to the repository-owned build when practical.

Shared vocabulary

Term	Meaning
project	one buildable unit with configuration and outputs
module	a separately modeled component inside or beside a larger build
task / goal	executable unit of build work
lifecycle	ordered stages with defined meanings
dependency	an input produced elsewhere or by another module
repository	service or directory that stores dependency artifacts and metadata
artifact	a produced file such as a JAR, sources JAR, POM, or module metadata
coordinate	stable artifact identity, commonly group, name, and version
plugin	extension that contributes build behavior
toolchain	selected JDK or other tool used by build work

The word repository is overloaded. A Git repository stores source history. An artifact repository stores packages such as JARs and their metadata. Maven's local repository is also not a source-control repository.

Maven and Gradle at a glance

Maven starts from a Project Object Model, conventions, packaging, lifecycles, phases, and plugin goals. A command such as `mvn verify` requests a lifecycle phase, so Maven runs preceding phases and their bound goals.

Gradle starts from settings, projects, plugins, a model of tasks, and task dependencies. A command such as `./gradlew build` selects a task; Gradle initializes the build, configures the required model, creates a task graph, and executes scheduled work.

TEXT

```
Maven                           Gradle
POM + packaging                 settings + build scripts
       |                                |
       v                                v
lifecycle phases                projects and task graph
       |                                |
       v                                v
plugin goals                    task actions
       +---------------+------------------+
                       v
               verified artifacts
```

Neither tool is universally superior. Maven often gives a predictable convention-heavy model. Gradle offers richer modeling and customization, which can improve large builds but also permits more accidental complexity.

What an SDE-2 engineer is expected to know

An entry-level answer may show how to add a dependency. An SDE-2 answer should also explain:

- which compile, runtime, and test classpaths change;
- how transitive version conflicts are resolved;
- why the same build can differ across machines;
- how unit and integration checks are separated;
- how module boundaries affect compilation and delivery;
- why cache reuse is correct or unsafe;
- how credentials and private repositories are isolated;
- how to diagnose the first meaningful cause in a long failure log.

The study route

```text
FOUNDATION
inputs -> work -> outputs -> artifacts -> repositories
   |
   v
TOOL BASICS
Maven POM/lifecycle <-> Gradle scripts/task graph
   |
   v
DEPENDENCIES AND TESTS
classpaths -> conflicts -> tests -> packaging
   |
   v
SCALE AND DELIVERY
modules -> toolchains -> CI -> cache -> publish -> security
   |
   v
SDE-2 JUDGMENT
diagnosis -> migration -> incident response -> interview defense
```

Read in order the first time. Every later chapter assumes that a build is a graph with explicit inputs and outputs, not a magic command.

First inspection habit

When joining a Java repository, do not begin by globally upgrading plugins or deleting caches. Inspect:

```bash
git status --short --branch
find . -maxdepth 2 -name pom.xml -o -name settings.gradle.kts \
  -o -name settings.gradle -o -name build.gradle.kts \
  -o -name build.gradle
java --version
```

Then look for `mvnw` or `gradlew`. A checked-in wrapper is the project's declared entry point. Read its properties and repository policy before executing downloaded code.

Interview drill

Question: Why do we need Maven or Gradle if `javac` and `jar` already exist?

Strong answer: `javac` and `jar` perform important individual operations. A build tool models the repeatable graph around them: source discovery, dependency resolution, generated inputs, multiple classpaths, tests, packaging, metadata, publication, incremental work, and team-wide configuration. The value is the declared and verifiable process, not merely fewer command-line characters.

WATCH OUT | Common mistakes

- Saying Maven is only dependency management or Gradle is only a faster Maven.
- Treating `clean` as a correctness requirement for every build.
- Assuming a successful compile proves runtime dependencies are present.
- Depending on an IDE's private configuration.
- Using the globally installed tool when a wrapper is available.
- Calling an artifact cache a source of truth.

Practice

1. **Foundation:** Draw inputs, work, and outputs for a one-class Java application.
2. **Foundation:** Explain the two meanings of repository used in this chapter.
3. **Interview Core:** Name three reasons a build can work locally and fail in CI.
4. **Debugging:** A teammate says, "Deleting everything fixed it." List the evidence still missing.
5. **SDE-2 Follow-up:** Define a minimal build contract for a library consumed by ten services.

Readiness check

- [ ] I can explain the build without naming either tool.
- [ ] I distinguish Git, local artifact, and remote artifact repositories.
- [ ] I know why wrappers, JDK selection, and dependency graphs belong to correctness.

SDE-2 CORE CHAPTER

Chapter 2 - Your First Java Build, Side by Side

This chapter builds the same small Java application with Maven and Gradle. The goal is not to memorize both files. It is to map the same responsibilities: identity, Java version, source layout, compilation, tests, and packaging.

Shared project

Use this structure for both versions:

TEXT

```
pricing-cli/
  src/main/java/com/example/pricing/PriceCalculator.java
  src/test/java/com/example/pricing/PriceCalculatorTest.java
  pom.xml                          # Maven version
  settings.gradle.kts             # Gradle version
  build.gradle.kts                # Gradle version
```

Maven and Gradle's Java plugins both understand the conventional `src/main/java` and `src/test/java` directories. Conventions remove configuration only when the project follows them.

JAVA

```java
package com.example.pricing;

public final class PriceCalculator {
    public long total(long unitPriceInCents, int quantity) {
        if (unitPriceInCents < 0 || quantity < 0) {
            throw new IllegalArgumentException("values must be nonnegative");
        }
        return Math.multiplyExact(unitPriceInCents, quantity);
    }
}
```

`Math.multiplyExact` makes overflow an explicit failure instead of silently wrapping. Build examples should still contain production-quality Java.

Maven version

`pom.xml` declares the project model:

XML

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>pricing-cli</artifactId>
  <version>1.0.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.13.0</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Build with the wrapper when the repository provides it:

BASH

```bash
./mvnw verify
jar tf target/pricing-cli-1.0.0-SNAPSHOT.jar
```

`verify` is a lifecycle phase. Maven reaches it by running earlier phases such as compile, test, and package. The JAR appears under `target/` by convention.

Gradle version

`settings.gradle.kts` gives the build and root project a stable name:

KOTLIN

```kotlin
rootProject.name = "pricing-cli"
```

`build.gradle.kts` applies Java behavior and configures compilation:

KOTLIN

```
plugins {
    java
}

group = "com.example"
version = "1.0.0-SNAPSHOT"

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

tasks.withType<JavaCompile>().configureEach {
    options.release = 21
    options.encoding = "UTF-8"
}
```

Build with the wrapper:

BASH

```
./gradlew build
jar tf build/libs/pricing-cli-1.0.0-SNAPSHOT.jar
```

`build` is a task, not a lifecycle phase. The Java plugin contributes tasks and relationships. Outputs appear under `build/` by convention.

Responsibility mapping

Responsibility	Maven	Gradle Kotlin DSL
project identity	POM coordinates	`group`, project name, `version`
Java behavior	compiler/plugin properties	Java plugin and toolchain
compile request	`mvn compile`	`./gradlew classes`
test request	`mvn test`	`./gradlew test`
full verification	`mvn verify`	`./gradlew check` or `build`
packaged JAR	`target/*.jar`	`build/libs/*.jar`
clean outputs	`mvn clean`	`./gradlew clean`

`build` depends on both checking and assembly in the conventional Gradle Java model. `check` focuses on verification. Prefer the narrowest command that matches the desired outcome.

Add JUnit Jupiter

Maven declares a test-scoped dependency and configures Surefire:

XML

```xml
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.11.4</version>
  <scope>test</scope>
</dependency>
```

Gradle declares it on the test implementation configuration:

KOTLIN

```kotlin
dependencies {
    testImplementation("org.junit.jupiter:junit-jupiter:5.11.4")
}

tasks.test {
    useJUnitPlatform()
}
```

The exact version is an example, not a promise of "latest." Production repositories should centralize and update versions under review.

What actually happened

On a first build, the tool may download plugins, metadata, and dependencies. It then compiles main source, compiles tests against a different classpath, runs tests, and creates a JAR. A second build may reuse local artifacts or skip work, but only when the tool can justify reuse.

TEXT

```
main source -> main classes ------------------> JAR
                    |                             |
test source ------+-> test classes -> tests --+-> verification result
```

Predict the output

If `PriceCalculatorTest` fails, will a normal `mvn package` or `./gradlew build` still publish a successful JAR result?

No. The JAR task may have produced a file before a later failure in some graphs, but the overall build fails. A file existing in an output directory is not proof that the build completed successfully or that it is safe to publish.

WATCH OUT | Common mistakes

- Running `mvn install` when only `verify` is needed; `install` mutates the local repository.
- Assuming Maven's `package` and Gradle's `build` are exact synonyms.
- Mixing an installed Gradle version with a checked-in wrapper.
- Setting only an IDE language level.
- Adding JUnit to the production runtime classpath.
- Treating generated directories as source-controlled inputs.

Interview drill

Question: What is the difference between `mvn package`, `mvn verify`, and `mvn install`?

Strong answer: All are phases in Maven's default lifecycle. Selecting a later phase runs earlier phases. `package` creates the distributable artifact, `verify` also completes verification checks such as integration-test result checks when configured, and `install` writes the artifact and POM to the local Maven repository for other local builds. I use `verify` for a CI quality gate unless local installation is intentionally required.

Practice

1. **Foundation:** Create both build files for the sample and identify every shared responsibility.
2. **Predict:** What directories are recreated after `clean`?
3. **Debugging:** The IDE compiles Java 21 syntax but CI rejects it. List the build files and JVMs to inspect.
4. **Interview Core:** Explain why `target/` or `build/` should usually be ignored by Git.
5. **SDE-2 Follow-up:** Define which command your pull-request CI should run and defend it.

SDE-2 CORE CHAPTER

Chapter 3 - Maven: Model, Lifecycle, Plugins, and Effective Build

Maven is predictable when you separate four ideas: the POM describes a project, packaging contributes default lifecycle bindings, phases name build stages, and plugin goals perform work.

The POM is input to an effective model

The `pom.xml` you read is not always the complete configuration Maven executes. Maven combines defaults, parent inheritance, dependency and plugin management, active profiles, user or installation settings, and command-line properties into an effective model.

TEXT

```
current POM + parent POMs + defaults + active profiles + settings
                                |
                                v
                        effective POM
                                |
                                v
                  lifecycle and plugin execution
```

Inspect rather than guessing:

BASH

```
./mvnw help:effective-pom -Dverbose -Doutput=effective-pom.xml
./mvnw help:effective-settings -Doutput=effective-settings.xml
./mvnw help:active-profiles
./mvnw help:describe -Dplugin=compiler -Ddetail
```

Effective files may expose repository locations or other environment details. Do not commit diagnostic output blindly.

Coordinates and packaging

The common identity is `groupId:artifactId:version`. Packaging defaults to `jar`; `pom` packaging is common for parents and aggregators. A coordinate names a component, while a classifier distinguishes an additional artifact such as `sources` under the same base coordinate.

Avoid encoding environments into artifact identity unless the release model truly requires different components. The same tested artifact should normally move across environments with runtime configuration supplied separately.

Lifecycles and phases

Maven has built-in `default`, `clean`, and `site` lifecycles. The most important default phases are:

```text
validate -> compile -> test -> package -> verify -> install -> deploy
```

When you run `mvn verify`, Maven does not start at `verify`; it traverses earlier phases. A phase can have zero or more goals bound to it. A goal is plugin work such as `compiler:compile` or `surefire:test`.

Direct goal execution is valid for diagnosis and one-off operations:

```bash
./mvnw dependency:tree
./mvnw help:effective-pom
```

For normal delivery, prefer a lifecycle phase that expresses the desired outcome.

Why mvn integration-test is usually the wrong endpoint

Integration-test infrastructure may be started in `pre-integration-test`, exercised in `integration-test`, stopped in `post-integration-test`, and checked in `verify`. Stopping at `integration-test` can bypass cleanup and final result checking. Invoke `verify` when this lifecycle is configured.

Plugins, executions, and configuration

A plugin contains goals. An execution can bind goals to phases and supply configuration:

```xml
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-enforcer-plugin</artifactId>
  <version>3.5.0</version>
  <executions>
    <execution>
      <id>enforce-build-contract</id>
      <phase>validate</phase>
      <goals><goal>enforce</goal></goals>
      <configuration>
        <rules>
          <requireJavaVersion><version>[21,22)</version></requireJavaVersion>
          <requireMavenVersion><version>[3.9,4)</version></requireMavenVersion>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>
```

Pin build-plugin versions. A build whose source dependencies are fixed but whose plugin versions drift is not fully controlled.

plugins versus pluginManagement

`build/plugins` activates or configures a plugin for the current project and inheriting children, subject to inheritance rules. `pluginManagement` centralizes defaults for a plugin but normally does not activate it by itself; a child still references the plugin under `plugins`.

The dependency equivalent is similar but not identical: `dependencyManagement` supplies versions and other managed values when a dependency is encountered. It does not automatically add the dependency to each module.

Parent versus aggregator

A parent relationship uses `<parent>` and inheritance. Aggregation uses `<modules>` in Maven 3 terminology. One POM can perform both roles, but the concepts differ:

- inheritance shares configuration down a parent chain;
- aggregation collects projects for one reactor build;
- a module can inherit from a parent that is not its aggregator;
- an aggregator can build a module that inherits elsewhere.

This distinction is a frequent interview question because large repositories often blur it.

Properties and profiles

Properties reduce duplication but can obscure ownership when overused. Profiles conditionally modify the model. Profile activation may depend on a property, JDK, operating system, file, or explicit flag.

Use profiles for genuine build variation, not for hiding environment-specific application configuration inside artifacts. Verify active profiles in incident diagnosis:

BASH

```
./mvnw help:active-profiles
./mvnw verify -Pquality-gates
```

Settings, mirrors, and credentials

Project-wide static configuration belongs in the repository or a shared parent. User and environment configuration belongs in `settings.xml`, including mirrors, proxies, and server credentials. A `<server>` ID must match the repository or deployment ID that requests credentials.

Never place repository passwords in `pom.xml`. Treat the local Maven repository as a cache plus locally installed artifacts, not a canonical deployment target.

Debugging sequence

When a Maven build surprises you:

1. reproduce with `./mvnw -version` and record the JDK;
2. run the narrow failing phase without unrelated flags;
3. inspect the first meaningful `Caused by` section, not only the last summary;
4. inspect active profiles and effective POM;
5. inspect dependency or plugin resolution separately;
6. use `-X` only after narrowing the problem because it is noisy and may reveal data;
7. compare local and CI settings, mirrors, toolchains, and environment inputs.

Interview traps

- A phase is not a plugin goal.
- `clean` belongs to a separate lifecycle; `mvn clean verify` requests two lifecycles in sequence.
- `install` does not deploy to a shared remote repository.
- `dependencyManagement` does not automatically add a dependency.
- `pluginManagement` does not normally execute the plugin.
- The child POM alone is not the effective build.
- Maven profiles are not Spring profiles.

Practice

1. **Foundation:** Explain phase, goal, plugin, and execution with one example each.
2. **Predict:** What earlier work runs for `mvn install`?
3. **Debugging:** A plugin executes only in CI. Which effective inputs do you compare?
4. **Interview Core:** Compare parent inheritance and reactor aggregation.
5. **SDE-2 Follow-up:** Design a company parent POM without forcing every plugin on every service.

Readiness check

- [ ] I can trace a phase to bound goals.
- [ ] I can obtain and interpret the effective POM.
- [ ] I know where project configuration, machine settings, and credentials belong.

SDE-2 CORE CHAPTER

Chapter 4 - Gradle: Model, Lifecycle, Tasks, and Lazy Configuration

Gradle is easiest to reason about when you distinguish build scripts from task execution. A build script configures a model; it is not a shell script that executes top to bottom as the build's work.

Files and responsibilities

File	Typical responsibility
`settings.gradle.kts`	build identity, projects, plugin and dependency repositories
`build.gradle.kts`	project plugins, dependencies, tasks, publications
`gradle.properties`	project or Gradle behavior without secrets in VCS
`gradle/libs.versions.toml`	dependency and plugin aliases and requested versions
`gradle/wrapper/*`	selected Gradle distribution contract
`build-logic/`	reusable convention plugins in a larger build

This book uses Kotlin DSL because it offers typed IDE support. You should still be able to recognize Groovy DSL in existing repositories.

Three lifecycle phases

Gradle's broad lifecycle is:

TEXT

```
INITIALIZATION              CONFIGURATION                EXECUTION
discover builds/projects -> register/configure model -> run scheduled tasks
settings.gradle(.kts)       build.gradle(.kts)           task actions
```

Initialization determines participating projects and included builds. Configuration applies plugins and creates the task graph for requested work. Execution runs scheduled tasks in dependency order.

A common mistake is performing network calls or expensive file scans during configuration. They run even when the selected task does not need the result and can prevent configuration-cache reuse.

Tasks and relationships

Tasks form a directed acyclic graph. Relationships have different meanings:

KOTLIN

```
val generateSchema by tasks.registering {
    outputs.file(layout.buildDirectory.file("schema/schema.json"))
    doLast {
        // Generate the declared output.
    }
}

tasks.named("classes") {
    dependsOn(generateSchema)
}
```

- `dependsOn` expresses a required predecessor.
- `mustRunAfter` constrains order only when both tasks are scheduled.
- `shouldRunAfter` is a softer ordering preference.
- `finalizedBy` schedules cleanup or follow-up after another task.

Do not use ordering where a true data dependency exists. If task B consumes task A's output, model the input/output relationship or dependency explicitly.

Inspect before changing:

BASH

```
./gradlew projects
./gradlew tasks --all
./gradlew help --task test
./gradlew build --dry-run
./gradlew :app:dependencies
```

Plugin contribution

Applying `java-library` adds source sets, configurations, tasks, and outgoing variants. It is not just a macro:

KOTLIN

```
plugins {
    `java-library`
}

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}
```

Prefer plugin IDs and published convention plugins over copy-pasted `allprojects` or `subprojects` blocks in large builds. Shared build logic should have tests, ownership, and versioning decisions like production code.

Task registration versus eager creation

Prefer lazy registration and named configuration:

KOTLIN

```
val verifyGeneratedFiles by tasks.registering {
    group = "verification"
    description = "Checks generated files without eager work"
}

tasks.named("check") {
    dependsOn(verifyGeneratedFiles)
}
```

Avoid eagerly traversing all tasks or calling `.get()` early without need. Lazy APIs let Gradle configure less work and support scalable builds.

Providers and declared inputs

Environment or file values used by tasks should be modeled lazily and declared:

KOTLIN

```
abstract class WriteBuildInfo : DefaultTask() {
    @get:Input
    abstract val revision: Property<String>

    @get:OutputFile
    abstract val outputFile: RegularFileProperty

    @TaskAction
    fun write() {
        outputFile.get().asFile.writeText(revision.get())
    }
}
```

Undeclared inputs cause stale up-to-date or cache hits. Undeclared outputs can collide across tasks. A fast wrong build is worse than a slow correct build.

doFirst, doLast, and configuration confusion

This code prints during configuration:

KOTLIN

```
println("configuring project")
```

This registers execution-time behavior:

KOTLIN

```
tasks.register("explain") {
    doLast {
        println("executing task")
    }
}
```

Interviewers often ask why code in a Gradle script runs even when a different task is requested. The answer is the configuration phase.

Build, configuration, and dependency caches

Do not merge these concepts:

- up-to-date checks reuse outputs already present in the current workspace;
- the build cache can restore task outputs from earlier compatible executions;
- the configuration cache can reuse the configured task graph;
- the dependency cache stores resolved artifacts and metadata.

Each has different keys, invalidation, and security boundaries.

Debugging sequence

BASH

```
./gradlew build --stacktrace
./gradlew build --info
./gradlew build --scan        # only under approved data policy
./gradlew dependencies
./gradlew dependencyInsight \
  --dependency jackson-databind \
  --configuration runtimeClasspath
```

Start with `--stacktrace`, then `--info`. `--debug` can expose credentials and is rarely the first move. Build scans can be powerful but may upload metadata to an external service; follow organizational policy.

Interview traps

- A Gradle build script is configuration code, not the task execution sequence.
- `build` is a task; `clean` is another task.
- `mustRunAfter` does not create a dependency.
- A version catalog does not force conflict-resolution outcomes by itself.
- `UP-TO-DATE` is not the same as `FROM-CACHE`.
- The daemon JVM and Java compilation toolchain can differ.
- Kotlin DSL does not make arbitrary custom build logic automatically safe or lazy.

Practice

1. **Foundation:** Draw initialization, configuration, and execution for `./gradlew test`.
2. **Predict:** Will `mustRunAfter` cause an otherwise unrequested task to execute?
3. **Debugging:** A task reuses stale output. List the missing input/output declarations to investigate.
4. **Interview Core:** Compare `UP-TO-DATE`, `FROM-CACHE`, and `SKIPPED`.
5. **SDE-2 Follow-up:** Decide when shared logic belongs in a convention plugin rather than the root script.

Readiness check

- **[ ]** I can explain why configuration code can run without task execution.
- **[ ]** I can distinguish task dependencies from ordering constraints.
- **[ ]** I treat providers, inputs, outputs, and lazy registration as correctness tools.

SDE-2 CORE CHAPTER

Chapter 5 - Dependencies, Classpaths, Scopes, and Configurations

Dependency declarations are not merely download instructions. They determine compile visibility, runtime contents, test isolation, transitive exposure, publication metadata, and sometimes consumer recompilation.

Start with classpaths

A typical Java project has at least these conceptual classpaths:

```text
main compile:   main classes + compile-visible dependencies
main runtime:   main classes + runtime dependencies
test compile:   test classes + main output + test compile dependencies
test runtime:   test classes + main output + test runtime dependencies
```

A dependency can be unnecessary at compile time but required at runtime, such as a database driver behind a standard API. A test library belongs on test classpaths, not the published production API.

Maven scopes

The common scopes are:

Scope	Main compile	Main runtime	Test	Transitive to consumer
`compile`	yes	yes	yes	yes
`provided`	yes	no	yes	no
`runtime`	no	yes	yes	yes
`test`	no	no	yes	no

Maven also defines `system`, which binds a build to a machine-local path and is strongly discouraged, and `import`, which is used only for POM dependencies in `dependencyManagement`.

XML

```xml
<dependencies>
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>${slf4j.version}</version>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Declare libraries used directly by source code even if they currently arrive transitively. That documents the module contract and prevents an unrelated upstream upgrade from silently removing the classpath.

Gradle configurations

Gradle configurations have roles: some accept dependency declarations, some are resolved as classpaths, and some expose variants to consumers. Java plugins create familiar declarable configurations:

- `implementation`: needed to implement the component but not exposed as its API;
- `api`: exposed to consumers by `java-library` when public signatures require it;
- `compileOnly`: compile-visible but absent at runtime;
- `runtimeOnly`: absent at compile time but present at runtime;
- `testImplementation` and `testRuntimeOnly`: test-specific inputs.

KOTLIN

```kotlin
plugins {
    `java-library`
}

dependencies {
    api("org.slf4j:slf4j-api:2.0.16")
    implementation("com.fasterxml.jackson.core:jackson-databind:2.18.2")
    runtimeOnly("org.postgresql:postgresql:42.7.5")
    testImplementation("org.junit.jupiter:junit-jupiter:5.11.4")
}
```

Do not declare on resolvable internal classpaths merely because the DSL allows low-level access. Use plugin-defined buckets and let the plugin model the graph.

API versus implementation

Suppose a library exposes `org.slf4j.Logger` in a public method signature. Consumers need SLF4J to compile against that API, so `api` is appropriate in Gradle. If Jackson is only used inside private implementation, use `implementation`.

TEXT

```
consumer compile classpath
       ^
       | exposed API dependency
library public surface ---- library implementation details
                                  |
                                  +-- hidden from consumer compile classpath
```

Overusing `api` expands consumer compile classpaths and increases recompilation. Hiding a required public type under `implementation` makes the published contract incomplete.

Maven's POM model does not express Gradle's API/implementation distinction with identical precision. Maven library authors must design published dependencies and public APIs carefully rather than assuming a one-to-one scope mapping.

Repositories and metadata

Dependency resolution uses both artifact files and metadata such as POMs or Gradle Module Metadata. Repository order, content, authentication, and availability affect the result.

Avoid adding arbitrary repositories until a missing artifact resolves. Multiple unfiltered repositories can create reliability and dependency-confusion risks. Prefer organization-controlled repository policy, HTTPS, and content restrictions.

Optional dependencies and exclusions

In Maven, an optional dependency is not automatically propagated to consumers. An exclusion removes a particular transitive path. In Gradle, exclusions and constraints can alter graph selection, while capabilities can model mutually exclusive implementations.

Use exclusions only with evidence. Removing a transitive dependency may compile and then fail at runtime. Add a deliberate replacement and a test that exercises the affected path.

NoClassDefFoundError reasoning

If compilation succeeds but runtime fails with `NoClassDefFoundError`:

1. identify the missing binary name;
2. inspect the runtime classpath, not only declared dependencies;
3. find whether scope/configuration omitted the artifact;
4. check packaging or container-provided assumptions;
5. check duplicate or incompatible versions;
6. reproduce outside the IDE.

The source may have compiled against a provided dependency that the deployment environment did not actually provide.

Interview drill

Question: Compare Maven `compile` scope with Gradle `implementation`.

Strong answer: They overlap for many application dependencies but are not exact synonyms. Maven `compile` is present on compile, runtime, and test classpaths and propagates to consumers. Gradle `implementation` is deliberately not exposed on a Java library consumer's compile classpath; `api` expresses that exposure. I compare the desired classpath and publication contract instead of memorizing a one-to-one mapping.

WATCH OUT | Common mistakes

- Adding every dependency to the broadest scope.
- Assuming a transitive dependency is a stable direct contract.
- Confusing dependency repositories with plugin repositories.
- Excluding a library without a runtime test.
- Treating version-catalog aliases as resolution locks.
- Assuming a successful compilation proves the packaged runtime classpath.

Practice

1. **Foundation:** Place JUnit, a JDBC API, a driver, and an internal JSON library on appropriate classpaths.
2. **Predict:** What happens when a `provided` dependency is absent in production?
3. **Debugging:** A public API exposes a type declared as Gradle `implementation`. Explain the consumer symptom.
4. **Interview Core:** Compare direct, transitive, optional, and excluded dependencies.
5. **SDE-2 Follow-up:** Design repository policy for public and private artifact namespaces.

Readiness check

- [ ] I reason from classpaths before choosing syntax.
- [ ] I understand Maven scope and Gradle configuration differences.
- [ ] I can diagnose compile-success/runtime-failure dependency problems.

SDE-2 CORE CHAPTER

Chapter 6 - Resolution Conflicts, BOMs, Platforms, and Locks

Declaring dependencies creates requests. Resolution turns those requests and transitive metadata into an actual graph. Interviews test whether you can explain why a particular version won and how you would verify it.

A conflict graph

TEXT

```
application
  +-- client-a:1.0 ----> json-core:2.1
  +-- client-b:1.0 ----> json-core:2.6
```

The build needs one compatible runtime result unless isolation is explicitly designed. "The latest wins" is not a universal answer.

Maven mediation

Maven commonly chooses the nearest definition in the dependency tree. A version reached through fewer dependency edges is nearer. At equal depth, declaration order can decide. A direct declaration is nearest, but scattering overrides across modules makes the graph hard to govern.

Inspect the selected and omitted paths:

BASH

```
./mvnw dependency:tree
./mvnw dependency:tree -Dverbose
./mvnw dependency:tree \
  -Dincludes=com.fasterxml.jackson.core:jackson-databind
./mvnw dependency:analyze
```

`dependencyManagement` centralizes versions when a dependency is encountered:

XML

```xml
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson</groupId>
      <artifactId>jackson-bom</artifactId>
      <version>${jackson.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

A BOM aligns a family of component versions. It does not prove the family is secure or mutually compatible with every other library in the application.

The Enforcer plugin can make convergence or banned-version rules explicit. Decide whether strict convergence is appropriate; forcing every path to one version can expose incompatible consumers rather than magically fixing them.

Gradle resolution

Gradle builds a graph, selects components, chooses compatible variants using attributes and capabilities, and then resolves artifacts. A simple version conflict often selects a newer candidate, but constraints, platforms, strict versions, rejects, forced rules, locks, and metadata can change the outcome.

Use evidence:

BASH

```bash
./gradlew :app:dependencies --configuration runtimeClasspath
./gradlew :app:dependencyInsight \
  --dependency jackson-databind \
  --configuration runtimeClasspath
```

`dependencyInsight` explains selection reasons and paths; it is more useful than reading only the declaration.

Catalogs, platforms, constraints, and locks

These mechanisms solve different problems:

Mechanism	Primary job
version catalog	central names and requested versions for build authors
platform / BOM	align or constrain a related version set
dependency constraint	express acceptable or preferred versions in a graph
strict version	reject outcomes outside an exact constraint
lock file	record resolved versions for repeatable future resolution
verification metadata	authenticate expected dependency bytes or signatures

A catalog is not a lock. A lock is not a checksum. A checksum is not a vulnerability assessment.

Gradle platform use:

KOTLIN

```
dependencies {
    implementation(platform("com.fasterxml.jackson:jackson-bom:2.18.2"))
    implementation("com.fasterxml.jackson.core:jackson-databind")
}
```

Locking example:

KOTLIN

```
dependencyLocking {
    lockAllConfigurations()
}
```

BASH

```
./gradlew dependencies --write-locks
./gradlew build
```

Review lock changes like code. A generated lock based on an already-compromised repository records the compromise faithfully.

Dynamic and changing versions

Selectors such as `1.+`, version ranges, and snapshots can change resolution without a source change. They may be useful for controlled compatibility testing, but release builds need a deliberate reproducibility policy.

Maven `SNAPSHOT` and Gradle changing modules also interact with metadata caches. `--refresh-dependencies` or deleting a cache is not a root-cause analysis; first determine what should have been immutable and which metadata expired.

Version skew failure walkthrough

Symptom: a service compiles but throws `NoSuchMethodError` after deployment.

Reasoning:

1. `NoSuchMethodError` usually means the runtime loaded a class version whose binary API differs from the one used at compile time.
2. Capture the exact class and method descriptor.
3. Inspect the resolved runtime graph and packaged contents.
4. Find duplicate JARs, container libraries, shaded copies, or dependency mediation.
5. Align versions or isolate incompatible components.
6. Add a packaging/runtime smoke test.

Do not "fix" this by adding the newest version blindly. The newest version may break a different path.

Interview drill

Question: Maven and Gradle resolve two versions of the same dependency. How do you know which one is used?

Strong answer: I do not infer solely from declaration order. In Maven I inspect `dependency:tree`, then apply nearest-definition and managed-version rules to the shown paths. In Gradle I inspect the target configuration with `dependencies` and `dependencyInsight`, because conflict selection, constraints, platforms, variants, and locks can affect the result. I verify the runtime classpath and packaged artifact when diagnosing a runtime linkage error.

Practice

1. **Foundation:** Distinguish requested, selected, and downloaded versions.
2. **Predict:** Can importing a BOM add every BOM component to the application?
3. **Debugging:** Trace a `NoSuchMethodError` from runtime symptom to dependency path.
4. **Interview Core:** Compare Maven nearest mediation with Gradle selection evidence.
5. **SDE-2 Follow-up:** Design a controlled dependency-upgrade and lock-update process.

Readiness check

- [ ] I can obtain selection evidence in both tools.
- [ ] I distinguish catalogs, BOMs/platforms, locks, and verification.
- [ ] I do not equate newest with compatible.

SDE-2 CORE CHAPTER

Chapter 7 - Testing, Integration Suites, and Quality Gates

A build should make different test purposes visible. Unit tests, component tests, integration tests, architecture checks, and end-to-end tests have different cost, isolation, and failure ownership.

Test pyramid as build topology

TEXT

```
many fast unit tests ---------------------- pull request
fewer component/integration tests -------- verify/check
small end-to-end or environment tests ---- delivery pipeline
```

The exact shape varies, but every gate needs a clear trigger, timeout, report, and owner.

Maven: Surefire and Failsafe

Surefire runs unit tests in the `test` phase. Failsafe is designed for integration tests across `integration-test` and `verify`, allowing `post-integration-test` cleanup before final failure reporting.

XML

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.5.2</version>
  <configuration>
    <useModulePath>false</useModulePath>
  </configuration>
</plugin>
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.5.2</version>
  <executions>
    <execution>
      <goals>
        <goal>integration-test</goal>
        <goal>verify</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

Naming conventions are configurable. Make them obvious, for example `*Test` for unit tests and `*IT` for integration tests. Run `./mvnw verify`, not only `integration-test`.

Useful distinctions:

- `-DskipTests` commonly skips running tests but may still compile them;
- `-Dmaven.test.skip=true` can skip both compilation and execution;
- exact behavior can be plugin-specific, so a release script should not hide skip flags.

Gradle unit tests

KOTLIN

```
dependencies {
    testImplementation("org.junit.jupiter:junit-jupiter:5.11.4")
}

tasks.test {
    useJUnitPlatform()
    failFast = false
    reports.junitXml.required = true
}
```

The Java plugin's `test` task contributes to `check`. An additional integration suite can be modeled explicitly. Gradle's JVM Test Suite API is documented as incubating in current releases, so label that boundary and consider a custom source set when API stability is required.

KOTLIN

```
testing {
    suites {
        val integrationTest by registering(JvmTestSuite::class) {
            useJUnitJupiter()
            dependencies {
                implementation(project())
            }
            targets.all {
                testTask.configure {
                    shouldRunAfter(tasks.test)
                }
            }
        }
    }
}

tasks.check {
    dependsOn(testing.suites.named("integrationTest"))
}
```

`shouldRunAfter` communicates order when both tasks are scheduled; `check` depending on the suite makes execution part of the gate.

Forking and parallelism

Parallel tests can reduce latency only when tests are isolated. Shared ports, static state, fixed database rows, wall-clock assumptions, and global files create nondeterminism.

Before increasing forks:

1. measure test duration and CPU/memory utilization;
2. classify tests by resource needs;
3. eliminate shared mutable state;
4. cap forks for CI container limits;
5. preserve per-test diagnostics;
6. compare failure and retry rates.

A retry plugin can reduce noise but can also normalize flaky behavior. Track the first-attempt failure rate and assign ownership.

Coverage and static analysis

Coverage is evidence of execution, not correctness. Quality gates can include formatting, compiler warnings, static analysis, forbidden APIs, architecture tests, mutation testing, dependency checks, and coverage thresholds.

Avoid one giant opaque "quality" task. Developers should be able to reproduce each failing gate locally and understand the report path.

Test environment inputs

Declare test inputs such as locale, timezone, encoding, environment variables, ports, and service endpoints. A test that reads undeclared machine state may pass locally, fail in CI, and poison caches.

TEXT

```
test result = code + test code + classpath + JVM + declared environment
```

Use temporary directories, random available ports, deterministic clocks, and isolated data. Do not log secrets while diagnosing test configuration.

Failure scenarios

Tests pass locally but not in CI

Compare wrapper version, launcher JDK, test JDK, locale, timezone, file-system case sensitivity, CPU/memory, parallelism, service availability, and active profiles/properties.

Integration environment is left running

Check whether Maven stopped at `integration-test` instead of `verify`, or whether Gradle cleanup was modeled only after successful task action rather than as a finalizer or external pipeline cleanup.

No tests executed

Verify discovery patterns, selected engine, JUnit Platform configuration, task filters, skip properties, and reports. A green task with zero discovered tests may need an explicit fail-on-no-tests policy.

Interview drill

Question: Why does Maven have both Surefire and Failsafe?

Strong answer: Surefire runs unit tests in the `test` phase and can fail immediately. Failsafe models integration tests across `integration-test` and `verify`; it preserves the lifecycle opportunity for `post-integration-test` teardown before final verification fails. This is why the normal endpoint is `mvn verify`.

Practice

1. **Foundation:** Assign unit and integration suites to both tools.
2. **Predict:** Does Gradle `shouldRunAfter` schedule the other task?
3. **Debugging:** A build reports success with zero tests. Create an evidence checklist.
4. **Interview Core:** Compare skip-test flags and explain release risk.
5. **SDE-2 Follow-up:** Design a 15-minute PR gate and a 60-minute post-merge gate.

Readiness check

- [ ] I can explain Surefire/Failsafe lifecycle behavior.
- [ ] I model Gradle suite execution as part of `check`.
- [ ] I measure flakiness instead of hiding it with retries.

SDE-2 CORE CHAPTER

Chapter 8 - Artifacts, Packaging, Metadata, and Runtime

Compilation produces class files. Packaging turns outputs into distributable components and metadata. Delivery still has to prove that the runtime starts with the intended classpath and configuration.

Common Java artifacts

- plain JAR containing project classes and resources;
- executable JAR with a main-class manifest and possibly bundled dependencies;
- WAR for a compatible servlet container;
- sources and Javadoc JARs for library consumers;
- POM and Gradle Module Metadata describing dependencies and variants;
- checksums, signatures, SBOMs, provenance, and test reports.

A plain JAR does not normally contain its dependency JARs. `java -jar app.jar` therefore needs an executable packaging strategy or a launcher/classpath arrangement.

Inspect before assuming

BASH

```
jar tf target/pricing.jar
unzip -p target/pricing.jar META-INF/MANIFEST.MF
javap -classpath target/pricing.jar \
  com.example.pricing.PriceCalculator
```

For Gradle, substitute `build/libs/...`. Inspect duplicate files, service descriptors, manifest entries, module descriptors, and dependency contents when diagnosing packaging.

Maven packaging and plugins

`jar` packaging contributes default goals for compilation, tests, JAR creation, installation, and deployment. Plugins can attach additional artifacts:

XML

```xml
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-source-plugin</artifactId>
  <version>3.3.1</version>
  <executions>
    <execution>
      <id>attach-sources</id>
      <goals><goal>jar-no-fork</goal></goals>
    </execution>
  </executions>
</plugin>
```

Understand whether a plugin goal forks a lifecycle, attaches an artifact, replaces the main artifact, or merely writes an untracked file. Duplicate plugin declarations are invalid design and Maven 4 is stricter about them.

Gradle archives and application distribution

KOTLIN

```kotlin
plugins {
    application
}

application {
    mainClass = "com.example.pricing.Main"
}

tasks.jar {
    manifest {
        attributes["Main-Class"] = application.mainClass.get()
    }
}
```

The Application plugin can create start scripts and distributions with runtime libraries. A manifest main class alone does not copy dependencies into the JAR.

For libraries:

KOTLIN

```kotlin
java {
    withSourcesJar()
    withJavadocJar()
}
```

Fat JAR and shading trade-offs

Bundling dependencies can simplify deployment but creates new obligations:

- merge service-provider files correctly;
- handle duplicate resources and licenses;
- avoid accidentally bundling signatures that no longer verify;
- decide whether packages must be relocated to prevent conflicts;
- record the true component inventory;
- verify startup and reflection-heavy paths.

Do not assume the first JAR task that includes `runtimeClasspath` is a correct production assembly.

Reproducible output

Bit-for-bit reproduction can be broken by timestamps, file order, permissions, generated paths, hostnames, usernames, locale, or differing JDK/plugin versions.

Maven plugins can honor `project.build.outputTimestamp`:

XML

```xml
<properties>
  <project.build.outputTimestamp>
    2026-01-01T00:00:00Z
  </project.build.outputTimestamp>
</properties>
```

Gradle archive tasks expose reproducibility controls, and modern defaults improve ordering and timestamp behavior. Still verify the actual artifact twice in isolated environments; configuration intent is not proof.

BASH

```bash
shasum -a 256 build/libs/*.jar
```

The same hash proves identical bytes, not that the bytes are safe or correct.

Runtime smoke testing

A useful release gate starts the packaged artifact, not just classes from the IDE or test classpath. Check:

- main class and launch command;
- required dependencies and native libraries;
- Java runtime compatibility;
- configuration and secret injection;
- health or readiness behavior;
- graceful failure on missing configuration;
- artifact identity reported in logs or metadata.

Interview drill

Question: Why can a build pass tests but the JAR fail at startup?

Strong answer: Tests may run against the tool-managed test runtime classpath, while the deployed JAR is a plain artifact without dependencies or has different packaging. The manifest may be wrong, a runtime-only dependency may be absent, duplicate resources may be merged incorrectly, or the runtime JDK may be incompatible. I inspect the artifact and run a packaged smoke test with the deployment-style command.

Practice

1. **Foundation:** Explain classes, plain JAR, executable distribution, and publication metadata.
2. **Predict:** Does adding `Main-Class` automatically embed dependency JARs?
3. **Debugging:** A service provider works in tests but disappears from a fat JAR. What do you inspect?
4. **Interview Core:** Separate reproducibility, integrity, and vulnerability evidence.
5. **SDE-2 Follow-up:** Define release checks for a reusable Java library.

Readiness check

- [ ] I inspect packaged contents rather than inferring them.
- [ ] I understand executable packaging and shading trade-offs.
- [ ] I include deployment-style smoke tests in release reasoning.

SDE-2 CORE CHAPTER

Chapter 9 - Multi-Module and Multi-Project Build Design

Splitting a repository into modules can enforce ownership and dependency direction, enable reuse, and reduce the affected build graph. It can also create ceremony, cyclic pressure, and accidental coupling. Module count is not architecture quality.

Example architecture

TEXT

```
pricing-parent
  +-- pricing-domain       pure domain types and rules
  +-- pricing-storage      persistence adapter -> domain
  +-- pricing-service      application -> domain + storage
```

Desired direction:

TEXT

```
service ---> storage ---> domain
    +---------------------> domain
```

A cycle such as domain depending on storage signals a boundary problem. Build tools can reject the cycle, but architecture must resolve it.

Maven reactor

An aggregator POM collects modules:

XML

```
<packaging>pom</packaging>
<modules>
  <module>pricing-domain</module>
  <module>pricing-storage</module>
  <module>pricing-service</module>
</modules>
```

The reactor collects projects, determines build order from instantiated relationships, selects requested projects, and builds them. Declaration order is not a substitute for dependency declarations.

Useful selection flags:

```bash
./mvnw -pl pricing-service -am verify
./mvnw -pl pricing-storage -amd test
./mvnw -rf :pricing-storage verify
```

- `-pl` selects projects;
- `-am` also builds required reactor dependencies;
- `-amd` also builds reactor dependents;
- `-rf` resumes from a project after a failure.

Understand the trade-off: resuming after a source change or changed environment may reuse earlier outputs that should be rebuilt. Reproduce cleanly before release.

Parent and BOM structure

A parent POM can centralize plugin and dependency management. A BOM POM communicates dependency versions. Keeping these roles distinct may improve reuse, but a small repository may reasonably combine parent and aggregator roles.

Avoid a parent that activates every expensive plugin in every child. Put default versions under management and activate behavior through clear module conventions.

Gradle multi-project build

`settings.gradle.kts` includes projects:

```kotlin
rootProject.name = "pricing-platform"
include("pricing-domain", "pricing-storage", "pricing-service")
```

A module dependency is explicit:

```kotlin
dependencies {
    implementation(project(":pricing-domain"))
    implementation(project(":pricing-storage"))
}
```

Run a qualified task:

```bash
./gradlew :pricing-service:build
./gradlew :pricing-domain:test
```

Gradle schedules required producer tasks based on project dependencies. Do not implement module ordering with manual task-order hacks.

Shared Gradle build logic

Small builds can use a root script carefully. Larger builds benefit from convention plugins that define the organization's Java service or library contract.

TEXT

```
build-logic/                        included build
  src/main/kotlin/
    company.java-library.gradle.kts
pricing-domain/build.gradle.kts    applies convention plugin
```

`buildSrc` is convenient but globally affects the build when changed. An explicit included `build-logic` build is more flexible and scales better. Avoid cross-project configuration that eagerly reaches into every child.

Composite builds

A multi-project build contains subprojects in one build. A composite build includes independent builds and can substitute a published dependency with local source during co-development:

KOTLIN

```
includeBuild("../shared-observability")
```

This is powerful for testing a library and consumer together, but publication metadata can differ from source substitution. Always validate the released coordinates too.

Selective build correctness

Path-based CI that builds only "changed modules" can miss:

- downstream consumers;
- shared build logic;
- parent/BOM/catalog changes;
- generated schemas;
- runtime-only wiring;
- integration tests across modules.

The safe affected set is graph-based, not merely directory-based.

```text
changed module + transitive dependents + shared contract consumers
```

Use periodic full builds as a backstop, but do not let them become the only place integration failures appear.

Interview drill

Question: What decides module build order?

Strong answer: Real dependency relationships should decide it. Maven's reactor sorts collected projects using instantiated relationships such as project dependencies and build plugins, not simply the `<modules>` order. Gradle constructs task dependencies across project dependencies. Manual ordering is a warning that the build graph is not modeled correctly.

Practice

1. **Foundation:** Split a Java service into domain, adapter, and application modules.
2. **Predict:** Does Maven `dependencyManagement` create reactor build order by itself?
3. **Debugging:** A selective build misses a consumer failure after a BOM change. Explain why.
4. **Interview Core:** Compare parent, aggregator, multi-project, and composite build.
5. **SDE-2 Follow-up:** Design affected-module CI with a periodic full-build backstop.

Readiness check

- [ ] I model module direction explicitly.
- [ ] I can use reactor and qualified-task selection safely.
- [ ] I know why source substitution does not replace publication testing.

SDE-2 CORE CHAPTER

Chapter 10 - Wrappers, Toolchains, and Environment Control

"Java version" can mean at least four things: the JDK launching the build tool, the JDK compiling source, the language/API release target, and the runtime executing tests or the application. Treat them as separate evidence.

Repository-owned wrappers

The Maven Wrapper and Gradle Wrapper let a repository declare the build-tool distribution used by developers and CI.

TEXT

```
developer / CI
      |
      v
wrapper script -> wrapper properties -> verified distribution -> build
```

Commit the expected wrapper files. Review distribution URL, version, checksum capability, and wrapper code changes. A wrapper downloads and executes tooling, so it is part of the supply chain.

Common entry points:

BASH

```
./mvnw --version
./mvnw verify
./gradlew --version
./gradlew build
```

Do not silently fall back to globally installed `mvn` or `gradle` in CI when a wrapper fails.

Maven toolchains

Maven itself runs on one JVM. Toolchain-aware plugins can select another JDK for compilation, tests, Javadoc, or signing. Machine installations can be mapped in `~/.m2/toolchains.xml`, while project configuration requests capabilities.

XML

```xml
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-toolchains-plugin</artifactId>
  <version>3.2.0</version>
  <executions>
    <execution>
      <goals><goal>select-jdk-toolchain</goal></goals>
    </execution>
  </executions>
  <configuration>
    <version>21</version>
  </configuration>
</plugin>
```

Verify which plugins are toolchain-aware. A plugin that ignores the selected toolchain may still use the Maven launcher JVM.

Gradle toolchains

KOTLIN

```kotlin
java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}
```

Toolchains can select or provision a compatible JDK according to configured resolver policy. The Gradle daemon or launcher JVM can differ from the compiler/test launcher selected by tasks.

Inspect:

BASH

```bash
./gradlew --version
./gradlew javaToolchains
./gradlew compileJava --info
```

--release versus source and target

`javac --release 17` constrains language features, bytecode target, and supported Java SE API signatures for release 17. Setting only source and target can still compile against APIs from the newer JDK that will not exist on the older runtime.

Maven:

XML

```xml
<maven.compiler.release>21</maven.compiler.release>
```

Gradle:

KOTLIN

```kotlin
tasks.withType<JavaCompile>().configureEach {
    options.release = 21
}
```

A toolchain answers "which JDK performs work?" Release answers "which Java platform API and class-file target may this compilation use?" Many projects set both deliberately.

Environment control

Potential inputs include:

- JDK vendor and patch version;
- build-tool and plugin versions;
- locale, timezone, and file encoding;
- operating system and architecture;
- environment variables and system properties;
- user Maven/Gradle configuration;
- repository mirrors and credentials;
- network and artifact-repository state.

Not every difference must be eliminated, but every correctness-relevant difference must be controlled, declared, or tested in a matrix.

"Works on my machine" diagnostic

Capture side-by-side evidence:

```bash
./mvnw --version
./gradlew --version
java --version
locale
env | sort                # redact secrets before sharing
```

Then compare wrapper properties, toolchains, active Maven profiles, Gradle properties, repository configuration, and clean-checkout behavior. Do not paste raw environment dumps into tickets; they can contain secrets.

Compatibility matrix

A library that publishes Java 17 bytecode but tests only on Java 21 has not demonstrated Java 17 runtime compatibility. Conversely, running the build tool on Java 17 does not prove the application artifact targets Java 17.

Define separate jobs when needed:

```text
build launcher JDK 21 -> compile --release 17 -> test runtime 17 and 21
```

Interview drill

Question: If Maven runs on JDK 21, does that mean the project produces Java 21 bytecode?

Strong answer: No. The launcher JVM, compiler JDK/toolchain, compiler release, and test/runtime JVM are separate. I inspect Maven version output, toolchain selection, compiler configuration, and the produced class-file target. With `--release`, I can constrain both bytecode and supported platform APIs.

Practice

1. **Foundation:** Name the four Java-version roles.
2. **Predict:** Can a Gradle daemon on one JDK compile with another toolchain?
3. **Debugging:** CI reports "invalid target release." Build an evidence sequence.
4. **Interview Core:** Explain why wrappers belong in version control.
5. **SDE-2 Follow-up:** Design a compatibility matrix for a Java 17 library maintained on JDK 21.

Readiness check

- [ ] I do not collapse launcher, toolchain, release, and runtime into one version.
- [ ] I treat wrapper changes as executable supply-chain changes.
- [ ] I can reproduce environment differences without leaking secrets.

SDE-2 CORE CHAPTER

Chapter 11 - CI, Performance, Caching, and Build Observability

A reliable build is fast enough to run frequently and transparent enough to debug. Optimize measured critical paths without weakening the build contract.

CI stages

A practical Java pull-request pipeline may separate:

TEXT

```
checkout + wrapper validation
          |
          v
compile + unit tests + static checks
          |
          v
integration tests + package smoke test
          |
          v
dependency/security evidence + artifact upload
```

Fail fast on cheap deterministic checks, but retain useful reports from later independent jobs. Cache only data whose trust and invalidation model is understood.

Maven performance model

Maven reactor builds can use parallelism, but plugin thread safety and resource contention matter:

BASH

```
./mvnw -T 1C verify
```

This requests a thread count relative to CPU cores. Measure it. Tests may already fork JVMs, and combined build/test parallelism can exhaust memory or database capacity.

The local repository avoids repeated downloads, but sharing an unscoped mutable local repository across concurrent CI jobs can create corruption or cross-job contamination. CI caches should be keyed to operating system, wrapper/tool version, and relevant dependency metadata; they should not replace artifact publication.

Maven Daemon is a separate tool with different operational assumptions. Label its use rather than implying every `mvn` process is persistent.

Gradle work avoidance

Gradle exposes several independent mechanisms:

1. **Up-to-date checks** skip a task when its declared inputs and local outputs match.
2. **Incremental task action** processes only changed input portions when supported.
3. **Build cache** restores task outputs from a compatible previous execution.
4. **Configuration cache** reuses the configured task graph when configuration inputs match.
5. **Daemon** reuses a JVM process and warmed state.
6. **Parallel execution** schedules independent project work concurrently.

BASH

```
./gradlew build --console=verbose
./gradlew build --build-cache
./gradlew build --configuration-cache
./gradlew build --parallel
```

Enable features deliberately, observe warnings, and fix incompatible custom tasks rather than switching to warn mode forever.

Cache correctness

A cache key is a statement: these declared inputs completely determine these outputs. Missing input example:

TEXT

```
task declares: source files
task reads:    source files + environment variable REGION
cache key:     excludes REGION
result:        output for us-east may be reused in eu-west
```

Secrets should rarely influence cacheable output. If they do, avoid storing secret-derived outputs in a shared cache and understand whether secret values enter keys, metadata, or logs.

Remote caches require write-trust policy. A compromised writer can poison outputs consumed by many developers. Common patterns let trusted CI push while developer machines read only.

Build timing method

Do not optimize from one warm laptop run.

1. define clean and incremental scenarios;
2. collect several runs on representative machines;
3. separate dependency download, configuration, compilation, tests, packaging, and upload;
4. find the critical path, not just the longest individual task;
5. inspect cache hit reasons and misses;
6. change one variable;
7. compare median and tail latency plus failure rate.

Build scans or profiles can help, but external telemetry must follow source, path, environment, and dependency-data policy.

Selective execution

Skipping modules or tests can accelerate feedback when selection is graph-correct. It is unsafe when change impact is based only on file paths. Shared parents, version catalogs, convention plugins, schemas, and dependency constraints can affect the entire build.

Maintain layers:

- quick local loop;
- required PR graph;
- post-merge broader graph;
- scheduled clean and compatibility builds;
- release build from an immutable commit.

Diagnosing a slow build

Ask:

- Is time spent downloading, configuring, compiling, testing, packaging, or waiting?
- Which work is on the critical path?
- Are tasks actually cacheable or merely expected to be?
- Which input changes cause misses?
- Is parallelism causing memory pressure and garbage collection?
- Are integration resources serialized?
- Did a build-logic change invalidate all projects?
- Is CI restoring a cache after the job already needs the data?

Interview drill

Question: What is the difference between Gradle's build cache and configuration cache?

Strong answer: The build cache stores outputs of cacheable task executions keyed by declared inputs and implementation details. The configuration cache stores the configured task graph and relevant configuration inputs so Gradle can skip configuration. They optimize different phases and can be used independently. Neither excuses undeclared inputs.

Practice

1. **Foundation:** Classify dependency cache, up-to-date output, and published artifact.
2. **Predict:** Can more parallel forks make a build slower?
3. **Debugging:** A cache hit returns region-specific stale output. Identify the contract defect.
4. **Interview Core:** Compare Gradle cache layers and Maven local-repository reuse.
5. **SDE-2 Follow-up:** Design trusted remote-cache read/write policy.

Readiness check

- [ ] I optimize measured critical paths.
- [ ] I distinguish work avoidance mechanisms.
- [ ] I treat cache correctness and writer trust as production concerns.

SDE-2 CORE CHAPTER

Chapter 12 - Publishing, Versioning, and Private Repositories

Publishing turns a local artifact into a component contract for other builds. It requires stable identity, correct metadata, credentials, immutability policy, and consumer validation.

Coordinates and version policy

Maven-compatible coordinates commonly contain:

```text
groupId : artifactId : version : packaging : classifier
```

Choose names that remain meaningful after teams reorganize. Avoid republishing different bytes under the same release coordinate. Mutable snapshots can support integration, but consumers must understand cache and reproducibility effects.

Semantic versioning can communicate compatibility intent, but it is not automatic proof. Java binary, source, behavior, serialization, schema, and configuration compatibility can change differently.

Maven install versus deploy

- `install` writes artifact and POM to the local repository;
- `deploy` uploads to the configured remote repository;
- `distributionManagement` declares deployment targets;
- credentials belong in settings under a matching server ID.

```xml
<distributionManagement>
  <snapshotRepository>
    <id>internal-snapshots</id>
    <url>https://packages.example/snapshots</url>
  </snapshotRepository>
  <repository>
    <id>internal-releases</id>
    <url>https://packages.example/releases</url>
  </repository>
</distributionManagement>
```

XML

```xml
<!-- ~/.m2/settings.xml, supplied securely -->
<server>
  <id>internal-releases</id>
  <username>${env.REPOSITORY_USER}</username>
  <password>${env.REPOSITORY_TOKEN}</password>
</server>
```

Do not print the effective settings in public logs without redaction.

Gradle Maven publication

KOTLIN

```kotlin
plugins {
    `java-library`
    `maven-publish`
}

publishing {
    publications {
        create<MavenPublication>("library") {
            from(components["java"])
            pom {
                name = "Pricing Contracts"
                description = "Stable pricing API contracts"
            }
        }
    }
    repositories {
        maven {
            name = "internal"
            url = uri("https://packages.example/releases")
            credentials {
                username = providers.environmentVariable("REPOSITORY_USER").orNull
                password = providers.environmentVariable("REPOSITORY_TOKEN").orNull
            }
        }
    }
}
```

Provider-based environment access delays evaluation. Still prevent credentials from entering configuration-cache entries, logs, or committed properties.

`publishToMavenLocal` is useful for a narrow local experiment, but local publication can mask missing remote metadata or stale components. A composite build may be better for source co-development, while a temporary isolated repository is better for testing publication behavior.

Library publication checklist

- main JAR, sources JAR, and Javadoc JAR as policy requires;
- correct POM scope and Gradle variant metadata;
- license, SCM, developer, and project metadata for public publication;
- checksums/signatures under the repository policy;
- dependency constraints that do not over-constrain consumers;
- no internal repositories or credentials leaked into published metadata;
- reproducible artifact evidence;
- consumer smoke tests using the published form;
- immutable release coordinate.

BOMs and platforms as products

A Maven BOM or Gradle Java platform publishes a compatibility policy, not executable code. It should have release notes, tests for supported combinations, ownership, and upgrade strategy.

Publishing every internal library version through one giant BOM can create lockstep coupling. Group components that are genuinely tested together.

Snapshot and release incident

Symptom: CI passes against `1.4.0-SNAPSHOT`, but production artifact built later behaves differently from the tested candidate.

Cause: the snapshot coordinate was mutable, so dependency bytes changed between verification and release.

Correction:

1. identify exact hashes resolved in each build;
2. stop promoting the unverified artifact;
3. release immutable component versions;
4. rebuild the consumer from pinned inputs;
5. promote the already verified consumer artifact rather than rebuilding per environment.

Interview drill

Question: What is the difference between a source version and a published artifact version?

Strong answer: A source version identifies repository state such as a Git commit. A component version is part of artifact coordinates and its compatibility contract. A release process connects them with build instructions, toolchain, dependency graph, hashes, and provenance. Neither identifier alone proves which bytes reached production.

Practice

1. **Foundation:** Explain install, deploy, publish, and promote.
2. **Predict:** Can `publishToMavenLocal` prove remote consumers will resolve correctly?
3. **Debugging:** A release POM exposes an internal implementation dependency. Find the declaration error.
4. **Interview Core:** Explain why release coordinates should be immutable.
5. **SDE-2 Follow-up:** Design a library release candidate and consumer-validation flow.

Readiness check

- [ ] I separate local installation from remote publication.
- [ ] I test published metadata, not only project compilation.
- [ ] I can connect commit, artifact coordinate, hash, and deployment.

SDE-2 CORE CHAPTER

Chapter 13 - Security, Reproducibility, and the Build Supply Chain

Builds execute plugins, resolve external metadata, download code, access credentials, and produce deployable bytes. Build configuration is production security code.

Threat model

TEXT

```
source change ----+
plugin change ----+--> build runner --> artifact --> repository --> deployment
dependency bytes -+         ^               |
wrapper/toolchain +         |               +--> SBOM/provenance/signature
credentials -----------------+
```

Threats include dependency confusion, compromised repositories or publisher accounts, malicious plugins, tampered wrappers, poisoned caches, leaked credentials, mutable versions, and build scripts that execute untrusted input.

Repository policy

Prefer approved HTTPS repositories and organization mirrors. Keep public and private namespaces distinct. Restrict which repository may serve which groups when the tool supports it.

Gradle can centralize repositories and fail project-local additions:

KOTLIN

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        exclusiveContent {
            forRepository {
                maven("https://packages.example/internal")
            }
            filter {
                includeGroupByRegex("com[.]example([.].*)?")
            }
        }
        mavenCentral()
    }
}
```

Maven mirrors and repository-manager policy can route approved resolution. Mirror configuration needs careful `mirrorOf` reasoning; an overbroad or unreachable mirror can break plugin and dependency resolution for every project.

Dependency verification

Gradle verification metadata can check dependency and plugin artifacts and metadata using expected checksums or signatures:

```bash
./gradlew --write-verification-metadata sha256 help
./gradlew build
```

Bootstrapping records what was resolved at that moment. Verify initial values through an independent trusted source before treating them as a security baseline.

Maven relies on repository transport, signatures/checksums, repository-manager controls, plugins, and organizational policy. Pin plugin and dependency versions, restrict repositories, and generate verifiable component inventories.

Wrapper security

For Gradle, pin the distribution checksum in wrapper properties and validate the wrapper JAR. For Maven, review wrapper distribution URL/type and wrapper code. A pull request changing wrapper files deserves focused review because it changes code that runs before the project build.

Do not embed wrapper credentials in a committed distribution URL.

Secrets

- inject short-lived credentials through the CI secret mechanism;
- use least-privilege publication tokens;
- separate read and write credentials;
- do not place secrets in build scripts, POMs, checked-in properties, or command history;
- avoid debug logs that print headers, environment, or effective settings;
- rotate first if a secret is exposed; deleting history is not revocation.

SBOM, signature, provenance, and reproducibility

These provide different evidence:

Evidence	Question answered
SBOM	which components are reported in the artifact/build?
checksum	are these bytes identical to expected bytes?
signature	did the holder of a signing identity sign these bytes/metadata?
provenance/attestation	which declared process and inputs produced the artifact?
reproducible rebuild	can an independent environment recreate identical bytes?
vulnerability scan	are known advisories associated with reported components?

None alone proves absence of malicious behavior. Combine evidence and protect the systems that create it.

Reproducibility checklist

- wrapper and plugin versions pinned;
- JDK/toolchain and release target controlled;
- no dynamic release dependencies;
- resolved graph recorded or constrained;
- timestamps, file ordering, permissions, and generated paths controlled;
- locale/encoding/timezone behavior tested;
- clean isolated rebuild compared by hash;
- publication uses the verified artifact, not a new rebuild.

Incident: compromised dependency version

1. contain by blocking the coordinate/version and pausing affected releases;
2. identify all resolved graphs, artifacts, caches, and deployments containing it;
3. preserve logs, lock files, SBOMs, hashes, and repository audit records;
4. select or rebuild with a verified safe version;
5. invalidate poisoned caches under controlled procedures;
6. rotate secrets accessible to executed malicious code;
7. redeploy verified immutable artifacts;
8. add repository, verification, detection, and upgrade controls.

Interview drill

Question: If a dependency checksum matches, is the dependency safe?

Strong answer: The checksum proves byte identity against the expected digest. Safety depends on how the expected digest was established, whether the publisher and repository were trusted, whether the component is vulnerable or malicious, and how it executes in context. I combine verification, provenance, SBOM/scanning, repository policy, and runtime controls.

Practice

1. **Foundation:** Separate integrity, authenticity, provenance, and vulnerability evidence.
2. **Predict:** What happens if verification metadata is generated after repository compromise?
3. **Debugging:** A CI token appears in `--debug` output. State the first response.
4. **Interview Core:** Explain dependency-confusion defenses for an internal group.
5. **SDE-2 Follow-up:** Design read/write trust for repositories and build caches.

Readiness check

- [ ] I treat plugins and wrappers as executable dependencies.
- [ ] I rotate exposed credentials before history cleanup.
- [ ] I do not overclaim what a checksum, signature, SBOM, or scan proves.

SDE-2 CORE CHAPTER

Chapter 14 - Complex Build Failure Playbooks

SDE-2 diagnosis should separate symptom, evidence, containment, cause, correction, validation, and prevention. These scenarios combine tool mechanics with delivery judgment.

Playbook method

For every incident:

```text
1. Scope and contain
2. Capture exact command, commit, wrapper, JDK, and environment
3. Find the first meaningful failure
4. Inspect effective model and resolved graph
5. Form one falsifiable hypothesis
6. Make the smallest safe correction
7. Validate locally, in clean CI, and at artifact/runtime boundary
8. Add a guardrail without hiding future failures
```

Scenario 1: IDE green, wrapper red

Evidence: IDE compiler and test runner versions differ from the build; generated sources exist only in the IDE.

Response: reproduce with wrapper, inspect toolchain/release and generation phase/task, configure IDE delegation, and make generated inputs part of the build graph.

Interview follow-up: Why is committing generated source not automatically the best fix? It creates dual ownership and drift unless generation policy explicitly requires checked-in outputs.

Scenario 2: Clean build passes, incremental build fails

Likely cause: task or plugin has undeclared inputs/outputs, stale generated files, or order dependence.

Response: preserve the failing workspace, compare task outcomes, disable caches selectively for diagnosis, correct the producer/consumer relationship, then test clean and incremental sequences.

Scenario 3: Incremental build passes, clean build fails

Likely cause: hidden reliance on local repository installation, stale generated output, missing declared dependency, or an IDE-created file.

Response: reproduce in an empty checkout and isolated artifact cache, then declare or generate the missing input. Never make "run module A once" an onboarding step.

Scenario 4: NoSuchMethodError only in production

Evidence: compile and runtime classpaths differ, container adds a library, fat JAR contains duplicates, or conflict mediation selected an incompatible version.

Response: capture the loaded class origin, inspect runtime graph and artifact contents, align or isolate versions, and add packaged runtime smoke coverage.

Scenario 5: Maven parent change is ignored

Causes: child inherits a released remote parent rather than reactor parent, `relativePath` differs, effective POM comes from cache, or profile changes selection.

Response: inspect `help:effective-pom -Dverbose`, parent coordinate and `relativePath`, reactor selection, and local repository state. Do not install random snapshots until the parent relationship is understood.

Scenario 6: Gradle task runs during every build

Causes: missing output, changing input, task action always marked out of date, non-cacheable implementation, or output deleted by another task.

Response: use verbose task outcomes, inspect declared properties, isolate nondeterministic values, and confirm no overlapping outputs.

Scenario 7: Dependency lock update changes hundreds of entries

Causes: platform/BOM change, repository metadata drift, configuration coverage changed, or tool upgrade altered resolution.

Response: do not accept mechanical churn. Compare selection reasons, group intentional upgrades, review security/compatibility, and run consumer tests.

Scenario 8: Private dependency resolves from public repository

Risk: dependency confusion or namespace takeover.

Response: block release, inspect repository order and requested coordinates, restrict internal group to the private repository, verify bytes, rotate credentials if malicious code executed, and audit other builds.

Scenario 9: CI cache makes a removed dependency appear available

Cause: build uses a broad machine-level classpath, local publication, or stale generated artifact rather than a declared dependency. A proper Maven/Gradle dependency cache should not add an undeclared graph node by itself.

Response: run in an isolated home/cache, inspect effective classpaths, and remove hidden installation assumptions.

Scenario 10: Integration-test infrastructure leaks

Maven: pipeline invoked `integration-test` rather than `verify`, bypassing final lifecycle stages.

Gradle: cleanup was not a finalizer or external finally step.

Response: clean up resources first, then repair lifecycle/task modeling and add timeout plus ownership tags.

Scenario 11: Parallel build is slower and flaky

Cause: nested test/build parallelism oversubscribes CPU/memory, modules contend for ports or database locks, or task/plugin work is not thread-safe.

Response: measure utilization and critical path, reduce one layer of concurrency, isolate resources, and compare tail latency plus failure rate.

Scenario 12: Published library compiles in Gradle but not Maven

Cause: publication metadata lost variant semantics, POM scopes are wrong, or a dependency required by the public API is hidden.

Response: inspect generated POM and module metadata, test a clean Maven consumer and Gradle consumer, correct API exposure, and republish under a new immutable version.

Scenario 13: Wrapper upgrade breaks only CI

Causes: CI JDK is incompatible, distribution access is blocked, checksum changed, wrapper executable bit is missing, or removed behavior affects a plugin.

Response: compare wrapper files and CI launcher JDK, validate the distribution checksum, use an upgrade compatibility report, and separate tool upgrade from unrelated code.

Scenario 14: Reproducible build hashes differ by operating system

Causes: line endings, file permissions/order, generated absolute paths, locale, or platform-specific native artifacts.

Response: compare archives structurally, normalize intended metadata, isolate platform-specific outputs, and document the reproducibility boundary rather than claiming universal bit identity.

Scenario 15: Release used unreviewed plugin code

Cause: plugin version was dynamic/unpinned, plugin repository was broad, or Gradle build logic from an included build changed outside the reviewed diff.

Response: stop publication, preserve resolved plugin evidence, verify produced artifacts, pin and restrict plugins, and include build-logic ownership in review rules.

Scenario 16: Secret leaked through build diagnostics

Response order: revoke or rotate; contain log/artifact access; identify exposure duration and consumers; clean retained logs/caches; replace with masked short-lived credentials; add logging and permission controls. Rewriting Git or deleting the log alone is insufficient.

Scenario 17: Selective CI misses a downstream break

Cause: change impact ignored transitive consumers, shared build logic, parent/BOM/catalog changes, or generated contracts.

Response: expand the graph, run the full verification gate, correct affected-set rules, and add a regression case for the change type.

Scenario 18: Migration produces different artifacts

Cause: lifecycle/task mappings are only superficially equivalent. Resource filtering, test discovery, manifest, dependency scope, generated source order, or publication metadata differs.

Response: create a parity matrix; compare tests, classpaths, archive contents, metadata, runtime smoke behavior, and hashes where applicable before switching CI.

Incident communication template

```text
Impact:
Affected commits/artifacts:
Containment:
Known evidence:
Current hypothesis:
Next falsifying test:
Recovery decision:
Validation:
Preventive owner and due date:
```

Practice

1. **Interview Core:** Lead scenario 4 aloud in five minutes.
2. **Debugging:** Create commands and evidence for scenario 6.
3. **SDE-2 Follow-up:** Compare containment for scenarios 8 and 15.
4. **SDE-2 Follow-up:** Design a parity gate for scenario 18.
5. **Leadership:** Write a blameless post-incident action for scenario 17 with a measurable owner.

Readiness check

- [ ] I preserve evidence before deleting caches.
- [ ] I distinguish immediate containment from durable prevention.
- [ ] I validate source, graph, artifact, and runtime boundaries.

SDE-2 CORE CHAPTER

Chapter 15 - Maven or Gradle: Selection and Migration Strategy

Tool choice should follow constraints, team capability, ecosystem, and measurable build behavior. Popularity and syntax preference are weak decision criteria.

Comparison without slogans

Dimension	Maven tendency	Gradle tendency
core model	declarative POM and lifecycle	programmable model and task graph
convention	strong standard defaults	strong plugins plus flexible customization
build logic	XML configuration and plugins	Kotlin/Groovy DSL and plugin code
dependency model	scopes, mediation, management	configurations, variants, constraints, platforms
performance tools	reactor parallelism, local repository	incremental work, build/config caches, parallelism
large-build structure	reactor, parents, aggregators	multi-project, convention and composite builds
risk	difficult inherited/effective model	accidental eager or imperative build logic

These are tendencies, not guarantees. A disciplined Maven build can be complex; a disciplined Gradle build can be highly conventional.

Decision questions

1. What build types and languages must be modeled?
2. How much custom generation or variant behavior exists?
3. What does the team already operate well?
4. Which plugins are mature and maintained?
5. How important are fine-grained incremental and remote-cache behavior?
6. How many modules and repositories participate?
7. Which artifact metadata must consumers receive?
8. What security, reproducibility, and audit controls are mandatory?
9. Can the team own custom build logic as production code?
10. What migration and IDE cost is acceptable?

For a conventional Java service portfolio, Maven may minimize novelty. For a large polyglot or heavily generated build, Gradle's modeling and work avoidance may be valuable. Do not migrate a healthy build solely to obtain a cleaner-looking file.

Migration is semantic parity

Treat migration as a behavior-preserving system change:

TEXT

```
existing contract -> parity matrix -> parallel builds -> artifact comparison
        |                                      |
        +---------- rollback path <------------+
```

Inventory:

- source sets and generated sources;
- compile and runtime classpaths;
- unit/integration discovery and reports;
- resource filtering;
- manifest and archive contents;
- module graph and selective commands;
- toolchains and release target;
- quality plugins and thresholds;
- publications, POM scopes, variants, signatures;

- credentials and repository rules;
- cache, reproducibility, and CI behavior.

Maven to Gradle mapping

Do not translate XML elements line by line. First state the contract.

```text
Maven verify gate          -> Gradle check/build plus explicit integration suite
dependencyManagement BOM  -> platform/constraints and publication policy
parent pluginManagement   -> convention plugins and version policy
reactor modules           -> included Gradle projects
Failsafe lifecycle        -> explicit integration task/suite plus cleanup
```

Generated Gradle builds from automated conversion can bootstrap syntax, but they do not prove behavior or maintainable structure.

Gradle to Maven mapping

Variant-aware or custom task behavior may not map exactly to Maven scopes and lifecycle. Decide whether to simplify, publish classifiers, split modules, or retain Gradle. A lossy POM cannot communicate every Gradle metadata constraint to Maven consumers.

Parallel validation

During a controlled transition:

1. freeze unrelated build rewrites;
2. build the same commit with both tools;
3. compare resolved graphs and classpaths;
4. run identical test inventories;
5. compare JAR contents and metadata;
6. run consumer and deployment smoke tests;
7. compare clean/incremental CI time and failure rate;
8. keep one authoritative publication path;
9. document rollback;
10. remove the old tool only after an observation window.

Avoid publishing both tools' outputs to the same release coordinate unless byte and metadata identity are intentionally proven.

Migration metrics

- clean and warm build median and p95;
- cache hit rate with correctness sampling;
- flaky failure and retry rate;
- CI compute and queue time;
- developer setup time;
- number of custom plugins/scripts;
- artifact and dependency parity exceptions;
- mean time to diagnose build failures.

The fastest benchmark is not a win if incidents become harder to diagnose.

Interview drill

Question: Which is better, Maven or Gradle?

Strong answer: I would not choose without constraints. For a conventional Java service with strong Maven expertise and predictable lifecycle needs, Maven may reduce custom build ownership. For a large multi-project build needing fine-grained incremental work, variant modeling, and shared convention plugins, Gradle may justify its flexibility. I compare plugin maturity, team capability, security, reproducibility, performance evidence, publication requirements, and migration cost, then define an exit plan.

Practice

1. **Foundation:** Identify one strength and one risk of each model.
2. **Predict:** Can a line-by-line conversion prove artifact parity?
3. **Debugging:** A migrated build has the same tests but a different runtime POM. Find the missing parity dimension.
4. **Interview Core:** Recommend a tool for a 20-service conventional Java portfolio.
5. **SDE-2 Follow-up:** Write an RFC outline for a 200-module migration.

Readiness check

- [ ] I choose from constraints rather than popularity.
- [ ] I define migration through observable parity.
- [ ] I include rollback and an observation window.

SDE-2 CORE CHAPTER

Chapter 16 - Realistic Maven and Gradle Interview Rounds

Use these as spoken simulations. Pause after the interviewer line, answer aloud, then compare the strong answer and follow-up.

Round 1: Build fundamentals

Interviewer: What problem does a Java build tool solve beyond compilation?

Candidate: It defines the repeatable graph around compilation: source layout, generated inputs, dependency resolution, separate classpaths, tests, packaging, metadata, publication, and environment/toolchain rules. It lets developers and CI request an outcome and verify the same contract.

Follow-up: What is the graph's most important property?

Candidate: Work must have explicit dependencies, inputs, and outputs. Otherwise ordering, incrementality, caching, and selective execution can be wrong.

Round 2: Maven phase versus goal

Interviewer: Is `compile` a Maven command or plugin goal?

Candidate: In `mvn compile`, `compile` is a lifecycle phase. Maven traverses preceding phases and executes goals bound to them. `compiler:compile` is a specific plugin goal. A goal can also be invoked directly.

Follow-up: Why prefer `verify` in CI?

Candidate: It reaches packaging and configured integration/quality verification without mutating the local repository like `install` or publishing like `deploy`.

Round 3: Gradle lifecycle

Interviewer: Why does code in `build.gradle.kts` run when I request an unrelated task?

Candidate: Build scripts participate in configuration. Gradle initializes projects, evaluates configuration needed to construct the task graph, then executes selected tasks. Eager configuration or side effects can therefore run without the task action.

Follow-up: How do you improve it?

Candidate: Use lazy task registration, providers, convention plugins, declared inputs, and configuration-cache-compatible APIs. I measure configuration time before rewriting.

Round 4: Scope mapping

Interviewer: Is Maven `compile` identical to Gradle `implementation`?

Candidate: No. Both can represent a normal application dependency, but Maven compile dependencies propagate to consumers. Gradle `implementation` is hidden from a Java library consumer's compile classpath; `api` expresses public exposure. I choose from classpath and publication needs.

Round 5: Version conflict

Interviewer: Two libraries request different Jackson versions. Which wins?

Candidate: I inspect the actual graph. Maven normally uses nearest definition unless management or direct declarations control it. Gradle selection can involve conflict rules, constraints, platforms, variants, and locks. I use dependency tree or dependency insight and verify the runtime artifact.

Follow-up: Why not force the newest?

Candidate: Newest does not prove binary or behavioral compatibility for both callers.

Round 6: BOM versus lock

Interviewer: A BOM makes the build reproducible, correct?

Candidate: It aligns or manages a version family but does not necessarily record the complete resolved graph, authenticate bytes, pin plugins/toolchains, or control timestamps. Locks and verification solve different parts, and reproducibility must be tested.

Round 7: Tests

Interviewer: Why is `mvn integration-test` risky as the CI command?

Candidate: Failsafe's model uses post-integration-test for teardown and verify for final result checking. Stopping at integration-test can bypass cleanup or verification. I invoke `mvn verify`.

Follow-up: How do you model the equivalent in Gradle?

Candidate: An explicit integration suite/task, ordered after unit tests, wired into `check`, with cleanup as a finalizer or pipeline finally action.

Round 8: JDK versions

Interviewer: The build says Java 21. What do you verify?

Candidate: Wrapper/build-tool launcher JDK, selected compiler/test toolchains, `--release` or compatibility target, and deployed runtime. They can differ.

Round 9: Multi-module ordering

Interviewer: Should I list Maven modules in dependency order?

Candidate: Real dependencies should establish order. The reactor sorts collected projects using instantiated relationships. Declaration order is not an architecture mechanism. Gradle similarly uses task/project dependencies.

Round 10: Gradle caches

Interviewer: Explain `UP-TO-DATE` versus `FROM-CACHE`.

Candidate: Up-to-date means current workspace outputs already match declared inputs. From-cache means compatible outputs were restored from a build cache. Configuration cache is separate and reuses the configured task graph.

Follow-up: What makes cache reuse unsafe?

Candidate: Missing inputs, overlapping outputs, nondeterminism, secret-derived data, or an untrusted cache writer.

Round 11: Runtime linkage failure

Interviewer: CI passes, production throws `NoSuchMethodError`. Lead the diagnosis.

Candidate: I capture the exact binary signature and loaded class origin, compare compile and runtime graphs, inspect the packaged artifact and container libraries, identify duplicate or mediated versions, and correct alignment or isolation. Then I add a packaged runtime smoke test.

Round 12: Publishing

Interviewer: Why test both Maven and Gradle consumers of a library?

Candidate: Gradle Module Metadata can express variants that a generated Maven POM cannot represent exactly. Incorrect API exposure or scopes may work for one consumer and fail for the other. Clean consumer tests validate published metadata.

Round 13: Security

Interviewer: A checksum matches. Are we safe?

Candidate: We have byte identity relative to an expected digest. We still need confidence in how the digest was established, publisher/repository trust, vulnerability and malicious-code assessment, provenance, and execution controls.

Round 14: Secret leak

Interviewer: A token is printed in Maven debug logs. What first?

Candidate: Revoke or rotate the token, then contain access and assess exposure. Log deletion is not revocation. I replace it with short-lived least-privilege credentials and fix redaction/debug policy.

Round 15: Tool selection

Interviewer: Give me a one-minute Maven-versus-Gradle recommendation.

Candidate: I start with build shape, plugin maturity, team expertise, customization, module scale, performance evidence, publication semantics, and security/reproducibility requirements. Maven often lowers novelty for conventional services. Gradle can justify richer task, variant, and cache modeling at scale. I would prototype the critical path and record migration and exit costs.

Round 16: Build migration

Interviewer: The converted build is green. Can we switch?

Candidate: Green is necessary but not sufficient. I compare resolved classpaths, test inventory, generated source order, resources, archive contents, manifests, publication metadata, runtime smoke behavior, and performance. I run parallel builds, retain one publication authority, and keep rollback until an observation window passes.

Round 17: Slow build

Interviewer: Developers want more parallelism. What do you do?

Candidate: Measure clean and incremental critical paths, resource utilization, nested test forks, cache misses, and p95 latency. More parallelism can increase contention and flakiness. I change one layer and compare speed plus reliability.

Round 18: SDE-2 ownership

Interviewer: What distinguishes senior build ownership from knowing commands?

Candidate: I make the build contract explicit, keep feedback fast and trustworthy, connect source to immutable artifacts, secure dependencies and credentials, diagnose from evidence, design safe recovery, and leave measurable guardrails owned by the team.

Self-scoring rubric

Score each answer from 0 to 3:

- 0: incorrect or command-only;
- 1: correct definition without mechanics;
- 2: mechanics plus diagnostic evidence;
- 3: mechanics, trade-offs, recovery, and preventive control.

A strong SDE-2 rehearsal scores at least 42 of 54 without reading notes.

Practice

1. Record rounds 5, 10, 11, 13, and 16 aloud.
2. Limit each first answer to 90 seconds.
3. Add one clarifying question before scenario answers.
4. Name the exact command that would obtain evidence.
5. End each incident answer with validation and prevention.

Readiness check

- [ ] I can answer without saying "Maven is XML" or "Gradle is faster."
- [ ] I include classpath, artifact, runtime, and security evidence.
- [ ] I can lead an unfamiliar build incident with a controlled method.

SDE-2 CORE CHAPTER

Chapter 17 - Practice Workbook, Solutions, Command Reference, and Sources

Complete the tasks in a disposable repository. Predict the graph or output before running a command. Do not use a production artifact repository for practice.

TRY IT | Foundation exercises

1. Draw inputs, work, outputs, and publications for a Java library.
2. Create the same Java 21 project with Maven and Gradle Kotlin DSL.
3. Explain POM coordinates and Gradle project identity.
4. Compare Maven phase, goal, plugin, and execution.
5. Compare Gradle initialization, configuration, and execution.
6. Map main compile, main runtime, test compile, and test runtime classpaths.
7. Place JUnit and a database driver in appropriate scopes/configurations.
8. Inspect Maven effective POM and active profiles.
9. Inspect Gradle projects, tasks, and a dry-run task graph.
10. Build a plain JAR and inspect its manifest and contents.
11. Explain wrapper files to a new teammate.
12. Compare launcher JDK, toolchain JDK, release target, and runtime JDK.
13. Add a unit test and locate its XML report.
14. Explain why generated output directories are normally ignored by Git.
15. State when `clean` is diagnostically useful and when it wastes feedback time.

TRY IT | Interview-core exercises

16 Draw a transitive version conflict and predict Maven mediation.

17 Use Maven dependency tree to confirm the selected path.

18 Use Gradle dependency insight on one runtime configuration.

19 Compare BOM, platform, catalog, constraint, lock, and checksum.

20 Explain Maven `dependencyManagement` without saying it adds dependencies.

21 Explain Gradle `api` versus `implementation` from a consumer classpath.

22 Configure Maven Surefire and Failsafe and explain the verify endpoint.

23 Configure a Gradle integration suite and wire it into `check`.

24 Diagnose a green build that discovered zero tests.

25 Explain `NoClassDefFoundError` versus `NoSuchMethodError` build causes.

26 Split a service into three acyclic modules.

27 Select one Maven reactor module plus required upstream modules.

28 Run one qualified Gradle subproject task.

29 Explain parent inheritance versus aggregation.

30 Explain multi-project versus composite Gradle builds.

31 Compare plain JAR, application distribution, and fat JAR.

32 Create a packaged runtime smoke-test plan.

33 Explain `install`, `deploy`, `publish`, and promote.

34 Review a generated publication POM for API/runtime mistakes.

35 Explain why a local publication can hide repository defects.

TRY IT | SDE-2 exercises

36 Design a wrapper and JDK upgrade rollout across 50 repositories.

37 Define an affected-module CI algorithm for a parent/BOM change.

38 Create a cache trust model with CI-only writers.

39 Diagnose a Gradle task with stale cache hits.

40 Diagnose a Maven build that works only after local `install`.

41 Compare clean, warm, and incremental performance scenarios.

42 Design a dependency-confusion defense for internal coordinates.

43 Respond to a compromised build plugin.

44 Respond to a token printed in debug logs.

45 Define SBOM, checksum, signature, provenance, and reproducibility evidence.

46 Design immutable snapshot-to-release promotion.

47 Build a Maven-to-Gradle parity matrix.

48 Define rollback criteria for a build-tool migration.

49 Diagnose different archive hashes across operating systems.

50 Write an incident update for a production runtime linkage error.

Cumulative assessments

Assessment A: New service

Design a two-module Java service with Java 21, unit tests, integration tests, wrapper entry point, dependency version policy, packaged smoke test, and PR CI. Provide Maven and Gradle command paths.

Assessment B: Dependency incident

A private `com.example:payments-api` resolves from a public repository on one CI runner. Provide containment, evidence, correction, validation, and prevention.

Assessment C: Slow monorepo

A 180-module build takes 40 minutes. Developers propose path-only module selection and maximum parallelism. Produce a measurement and safe-optimization plan.

Assessment D: Library release

A Gradle-built library works for Gradle consumers but Maven consumers miss a type from a public signature. Diagnose metadata and publish a safe corrected release.

Assessment E: Migration

Plan Maven-to-Gradle migration for a service using generated OpenAPI source, unit and integration tests, shading, an internal BOM, and artifact signing.

Final readiness assessment

In 45 minutes, explain and sketch:

1. the shared build model;
2. Maven lifecycle and effective-model mechanics;
3. Gradle lifecycle, task graph, providers, and caches;
4. dependency selection and runtime linkage diagnosis;
5. tests, modules, artifacts, wrappers, toolchains, CI, publication, and security;
6. one production incident with containment and recovery;
7. a constraint-based Maven-versus-Gradle recommendation.

Solution sketches

Exercises 16-20

Draw every dependency path and distinguish requested from selected versions. Maven evidence comes from `dependency:tree`; Gradle evidence is configuration-specific and comes from `dependencies` plus `dependencyInsight`. A BOM/platform aligns versions, a catalog centralizes declarations, a lock records resolution, and a checksum verifies bytes.

Exercises 22-24

Surefire owns unit-test execution in Maven's test phase. Failsafe spans integration-test and verify so teardown can run before final failure. In Gradle, an integration suite must be scheduled by `check`; ordering alone is insufficient. Zero discovered tests require engine, naming/filter, source set, and report inspection.

Exercises 25 and 40

`NoClassDefFoundError` often signals missing runtime content. `NoSuchMethodError` signals binary mismatch between compile and runtime definitions. A build that needs local `install` likely has an undeclared project/artifact relationship; reproduce with an isolated local repository and declare it.

Exercises 37-39

Affected selection follows project dependencies, dependents, shared build logic, parents, platforms/catalogs, generated contracts, and integration edges. Cache correctness requires complete declared inputs/outputs and trusted writers. Preserve a stale-hit reproduction before invalidating caches.

Exercises 42-45

Restrict internal namespaces to an approved private repository, verify artifacts, and separate read/write identities. Rotate exposed secrets first. SBOM inventory, checksum identity, signature identity, provenance, reproducibility, and vulnerability evidence answer distinct questions.

Exercises 47-49

Migration parity covers graphs, test inventory, generated work, resources, artifacts, metadata, runtime, and performance. Roll back on correctness or publication mismatch. Different hashes require structural archive comparison for timestamps, order, paths, permissions, encodings, and platform-specific files.

Maven command reference

BASH

```
./mvnw --version
./mvnw test
./mvnw verify
./mvnw dependency:tree
./mvnw dependency:analyze
./mvnw help:effective-pom -Dverbose
./mvnw help:active-profiles
./mvnw -pl :module -am verify
./mvnw -pl :module -amd test
./mvnw -rf :module verify
```

Gradle command reference

BASH

```
./gradlew --version
./gradlew projects
./gradlew tasks --all
./gradlew test
./gradlew check
./gradlew build
./gradlew :module:build
./gradlew dependencies --configuration runtimeClasspath
./gradlew dependencyInsight --dependency name \
  --configuration runtimeClasspath
./gradlew build --console=verbose
./gradlew build --configuration-cache --build-cache
```

Official sources

Version-sensitive statements were checked on 2026-07-29. Consult current official documentation before changing production configuration.

Apache Maven

- Maven lifecycle: https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html
- POM model: https://maven.apache.org/pom.html
- Dependency mechanism: https://maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html
- Standard layout: https://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html
- Multiple modules: https://maven.apache.org/guides/mini/guide-multiple-modules.html
- Toolchains: https://maven.apache.org/guides/mini/guide-using-toolchains
- Reproducible builds: https://maven.apache.org/guides/mini/guide-reproducible-builds.html
- Surefire: https://maven.apache.org/surefire/maven-surefire-plugin/
- Failsafe: https://maven.apache.org/surefire/maven-failsafe-plugin/
- Effective POM: https://maven.apache.org/plugins/maven-help-plugin/effective-pom-mojo.html

Gradle

- Build lifecycle: https://docs.gradle.org/current/userguide/build_lifecycle.html
- Wrapper: https://docs.gradle.org/current/userguide/gradle_wrapper.html
- Java and Java Library plugins: https://docs.gradle.org/current/userguide/java_library_plugin.html
- Dependency management: https://docs.gradle.org/current/userguide/core_dependency_management.html
- Dependency locking: https://docs.gradle.org/current/userguide/dependency_locking.html
- Dependency verification: https://docs.gradle.org/current/userguide/dependency_verification.html
- JVM test suites: https://docs.gradle.org/current/userguide/jvm_test_suite_plugin.html
- Multi-project builds: https://docs.gradle.org/current/userguide/intro_multi_project_builds.html
- Composite builds: https://docs.gradle.org/current/userguide/composite_builds.html
- Build cache: https://docs.gradle.org/current/userguide/build_cache.html
- Configuration cache: https://docs.gradle.org/current/userguide/configuration_cache.html
- Maven publishing: https://docs.gradle.org/current/userguide/publishing_maven.html

Final checklist

- [ ] I can explain the concept before showing syntax.
- [ ] I can inspect effective configuration and resolved graphs.
- [ ] I can separate compile, test, package, publish, and deploy outcomes.
- [ ] I can diagnose across source, toolchain, graph, artifact, and runtime.
- [ ] I can defend build security, reproducibility, caching, and migration choices.

SDE-2 CORE CHAPTER

Chapter 18 - Dependency-Free Java 21 Build Graph Companion

This executable model validates dependency-first module ordering, affected-module selection, cycle detection, and deterministic cache-key inputs.

JAVA (continued 1/4)

```java
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

public final class BuildToolModel {
    private BuildToolModel() {
    }

    public static List<String> topologicalOrder(Map<String, Set<String>> dependencies) {
        Map<String, Integer> remaining = new LinkedHashMap<>();
        Map<String, Set<String>> dependents = new HashMap<>();
        for (String module : dependencies.keySet()) {
            remaining.put(module, 0);
        }
        for (Map.Entry<String, Set<String>> entry : dependencies.entrySet()) {
            for (String dependency : entry.getValue()) {
                if (!dependencies.containsKey(dependency)) {
                    throw new IllegalArgumentException("unknown module: " + dependency);
                }
                remaining.merge(entry.getKey(), 1, Integer::sum);
                dependents.computeIfAbsent(dependency, ignored -> new LinkedHashSet<>())
                        .add(entry.getKey());
            }
        }
```

JAVA (continued 2/4)

```
        Deque<String> ready = new ArrayDeque<>();
        remaining.entrySet().stream()
                .filter(entry -> entry.getValue() == 0)
                .map(Map.Entry::getKey)
                .sorted()
                .forEach(ready::addLast);

        List<String> order = new ArrayList<>();
        while (!ready.isEmpty()) {
            String module = ready.removeFirst();
            order.add(module);
            List<String> unlocked = new ArrayList<>(
                    dependents.getOrDefault(module, Set.of()));
            Collections.sort(unlocked);
            for (String dependent : unlocked) {
                int count = remaining.merge(dependent, -1, Integer::sum);
                if (count == 0) {
                    ready.addLast(dependent);
                }
            }
        }
        if (order.size() != dependencies.size()) {
            throw new IllegalArgumentException("module graph contains a cycle");
        }
        return List.copyOf(order);
    }

    public static Set<String> affectedModules(
            Map<String, Set<String>> dependencies,
            Set<String> changedModules) {
        Map<String, Set<String>> dependents = new HashMap<>();
        for (Map.Entry<String, Set<String>> entry : dependencies.entrySet()) {
            for (String dependency : entry.getValue()) {
                dependents.computeIfAbsent(dependency, ignored -> new HashSet<>())
                        .add(entry.getKey());
            }
```

JAVA (continued 3/4)

```
        }
        Set<String> affected = new LinkedHashSet<>(changedModules);
        Deque<String> pending = new ArrayDeque<>(changedModules);
        while (!pending.isEmpty()) {
            String current = pending.removeFirst();
            for (String dependent : dependents.getOrDefault(current, Set.of())) {
                if (affected.add(dependent)) {
                    pending.addLast(dependent);
                }
            }
        }
        return Set.copyOf(affected);
    }

    public static String cacheKey(Map<String, String> declaredInputs) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            declaredInputs.entrySet().stream()
                    .sorted(Map.Entry.comparingByKey())
                    .forEach(entry -> {
                        digest.update(entry.getKey().getBytes(StandardCharsets.UTF_8));
                        digest.update((byte) 0);
                        digest.update(entry.getValue().getBytes(StandardCharsets.UTF_8));
                        digest.update((byte) 0);
                    });
            return toHex(digest.digest());
        } catch (NoSuchAlgorithmException exception) {
            throw new IllegalStateException("SHA-256 is required by the JDK", exception);
        }
    }

    private static String toHex(byte[] bytes) {
        StringBuilder result = new StringBuilder(bytes.length * 2);
        for (byte value : bytes) {
            result.append(Character.forDigit((value >>> 4) & 0xf, 16));
            result.append(Character.forDigit(value & 0xf, 16));
```

JAVA (continued 4/4)

```
        }
        return result.toString();
    }

    public static void main(String[] arguments) {
        Map<String, Set<String>> graph = new LinkedHashMap<>();
        graph.put("domain", Set.of());
        graph.put("storage", Set.of("domain"));
        graph.put("service", Set.of("domain", "storage"));

        List<String> order = topologicalOrder(graph);
        if (!order.equals(List.of("domain", "storage", "service"))) {
            throw new AssertionError("unexpected build order: " + order);
        }
        Set<String> affected = affectedModules(graph, Set.of("domain"));
        if (!affected.equals(Set.of("domain", "storage", "service"))) {
            throw new AssertionError("unexpected affected set: " + affected);
        }

        String first = cacheKey(Map.of("source", "abc", "release", "21"));
        String reordered = cacheKey(Map.of("release", "21", "source", "abc"));
        String changed = cacheKey(Map.of("source", "abcd", "release", "21"));
        if (!first.equals(reordered) || first.equals(changed)) {
            throw new AssertionError("cache-key contract failed");
        }

        try {
            topologicalOrder(Map.of("a", Set.of("b"), "b", Set.of("a")));
            throw new AssertionError("cycle should fail");
        } catch (IllegalArgumentException expected) {
            // Expected.
        }
        System.out.println("BuildToolModel checks passed");
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: inputs, outputs, project layout, first builds, lifecycles, and classpaths
- Interview Core: dependency resolution, tests, artifacts, modules, wrappers, and toolchains
- SDE-2 Follow-up: CI performance, caches, publishing, supply-chain controls, incidents, and migration
- Readiness: complete 18 interview rounds, 50 workbook tasks, and five cumulative assessments

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: JAVA 02 - Git and GitHub for Java Engineers

Series index

Next book: JAVA 04 - Advanced Java B: Language, OOP, and Modern Java

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, or System Design and Backend first, then follow the books inside that segment in order. Stable study-step codes remain on filenames and links; the clearer segment book codes appear on the website and every PDF cover.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Advanced Java B: Language, OOP, and Modern Java
Java	**JAVA 05**	Advanced Java C: Collections, Streams, and I/O
Java	**JAVA 06**	Advanced Java A: JVM and Execution
Java	**JAVA 07**	Advanced Java D: Concurrency and the Memory Model
Java	**JAVA 08**	Advanced Java E: Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Advanced Java G: Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
System Design	**SD 01**	Advanced Java F: Design, Backend, Testing, and Security
System Design	**SD 02**	MySQL for Java Backend Interviews
System Design	**SD 03**	Hibernate and JPA for Java Backend Interviews
System Design	**SD 04**	Spring Framework for Java Engineers
System Design	**SD 05**	Spring Boot for Java Backend Engineers

SEGMENT	BOOK	FOCUSED LEARNING PATH
System Design	**SD 06**	Spring Data for Java Backend Engineers
System Design	**SD 07**	MongoDB for Java Backend Interviews
System Design	**SD 08**	Redis for Java Backend Interviews
System Design	**SD 09**	Apache Kafka and Spring Kafka
System Design	**SD 10**	Spring Ecosystem Extensions
System Design	**SD 11**	Spring AI for Java Engineers
System Design	**SD 12**	Advanced Java H: Spring Boot and REST APIs
System Design	**SD 13**	Advanced Java I: Persistence, SQL, JPA, and Caching
System Design	**SD 14**	Advanced Java J: Distributed Systems and System Design

Roadmap editions identify planned books whose chapter sets will be expanded one at a time. Existing deep-dive books remain available later in each segment, so no published material is displaced.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Java Engineering - Book 03 of 09 - Study Step 19B. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.